

Modul 02: Komponenten

<i>Ziel:</i>	<i>Erste Komponente erstellen</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~45min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>05.08.2024</i>

1 Neue Komponente erstellen

Komponenten sind die grundlegenden Bausteine einer Angular Applikation. Wir verwenden die Angular CLI um eine neue Komponenten zu erstellen. Im nächsten Schritt erstellen wir jeweils eine Komponente `TaskItem` und `TaskList`:

```
ng generate component task-item
```

Oder alternative Kurzschreibweise:

```
ng g c task-list
```

Es wurden nun vier neue Dateien erzeugt. Wenn unsere App nicht als Standalone konfiguriert wurde, so werden noch die Deklarationen in der Modul-Datei automatisch hinzugefügt.

Bei mehreren Modulen ist es daher wichtig darauf zu achten, dass man sich beim Erzeugen der Komponente im richtigen Verzeichnis befindet. Alternativ kann mit einem Schalter der Name des Moduls konfiguriert werden.

Changes		5
TS	app.module.ts todo-app\src\app	M
#	task-item.component.css todo-app\src\app\task-item	U
<>	task-item.component.html todo-app\src\app\task-item	U
TS	task-item.component.spec.ts todo-app\src\app\task-item	U
TS	task-item.component.ts todo-app\src\app\task-item	U

Wir sehen in der Datei `task-item.component.ts` eine Annotation, welche die Einzelnen Sourcen verknüpft. (Mit Ausnahme der `spec.ts` Datei, welche den Unit Test repräsentiert)

```
@Component({
  selector: 'app-task-item', // CSS-Element-Selektor der Komponente
  templateUrl: './task-item.component.html', // Pfad zum HTML-Template
  styleUrls: ['./task-item.component.css'], // Pfade zu privaten Stylesheets
})
export class TaskItemComponent {
}
```

2 Eigenschaften hinzufügen

Wir erstellen in der Code-Behind Datei (`task-item.component.ts`) ein Property welches wir der mustache (vgl. `mustache.js` als web template system) Schreibweise im HTML-Template binden können. Mit `@Input()` markieren wir das Property, damit wir von außen Eigenschaften übergeben können.

```
export class TaskItemComponent {
  @Input() title = 'Nicht angegeben'; // Standardwert
}
```

In der HTML Datei können wir mit doppelten geschweiften Klammern das öffentliche Property binden.

```
<span class="title">{{ title }}</span>
```

In der `task-list.component.html` können wir mittels dem Selektor die neue Komponente einbinden und auch die Eigenschaft von außen setzen.

```
<ul>
  <li><app-task-item title="Angular Kurs absolvieren" /></li>
</ul>
```

Natürlich muss in der Wurzel-Komponente (`app.component.html`) die `app-task-list` hinzugefügt werden. Den existierenden HTML-Code und die Styles verschieben wir in die entsprechenden Dateien von `app-task-list`.

3 Modell hinzufügen

Wir erstellen in VSCode im Ordner `src` eine neue Datei `lib/models/task-item.model.ts` (VSCode erstellt den Unterordner `models` automatisch). Unser Modell erstellen wir als Interface und sieht wie folgt aus.

```
export interface TaskItem {
  id: string;
  title: string;
  completed: boolean;
  priority: 'important' | 'default' | 'low';
  dueDate?: Date;
  labels?: string[];
}
```

Die `Title` Eigenschaft soll nun `item` heißen und vom Typ `TaskItem` sein.

```
item = { title: 'Nicht angegeben' } as TaskItem;
```

Natürlich muss das HTML-Template noch entsprechend angepasst werden. Wir müssen beachten, dass nur strings mit geschweiften Klammern `{{ }}` gebunden werden können. Andere Datentypen müssen mit eckigen Klammern `[ ]` an Attribute gebunden werden.

```
<div class="item">
  <input type="checkbox" [checked]="item.completed" />
  <span class="title" title="{{ item.id }}">{{ item.title }}</span>
</div>
```

Die entsprechende Style Klasse lautet dazu:

```
.item {
  display: grid;
  grid-template-columns: auto 1fr auto;
  grid-gap: 1rem;
}
```

In der `task-list.component.ts` müssen wir jetzt das Todo-Item im Code-Behind definieren.

```
export class TaskListComponent {
  taskList = [
    { title: 'Angular Kurs absolvieren' } as TaskItem
  ];
}
```

Statt der statischen Bindung mit der mustache-Syntax in geschweiften Klammern `{{ }}` müssen wir jetzt die Objekt-Instanz mit eckigen Klammern `[ ]` binden.

```
<ul>
  <li><app-task-item [item]="taskList[0]" /></li>
</ul>
```

4 Events hinzufügen

Ein Event wird mit runden Klammern () gebunden. Dem Button kann so ein Klick-Event bzw. dem Input-Feld ein Input-Event hinzugefügt werden. Nach dem Auslösen des Events soll eine Statusausgabe erfolgen, welche die Gesamtanzahl der Aufgaben ausgibt.

```
<form class="grid">
  <label for="">Aufgabe</label>
  <input type="text" (input.enter)="addTask($event, 'Irgendwas')" />
  <br />
  <button (click)="addTask($event, 'Irgendwas')">Neue Aufgabe</button>
</form>

<p class="status">{{ status }}</p>
```

Im Code-Behind fehlt noch die entsprechende Methode, welche eine Aufgabe erzeugt.

```
addTask(e: Event, title: string) {
  e.preventDefault();

  const newTask: TaskItem = {
    title,
    id: `${Date.now()}`,
    completed: false,
    priority: 'default',
  };

  this.taskList.push(newTask);
  this.setStatus(`${this.taskList.length + 1} Aufgaben insgesamt`);
}
```

5 Status ausgeben

Um die User Experience zu verbessern, können wir den Status von User Actions ausgeben.

```
// Property für Statusausgabe
status = '';
currentStatusId = 0;

setStatus(message: string) {
  this.status = message;

  if (this.currentStatusId) {
    clearTimeout(this.currentStatusId);
  }
  this.currentStatusId = setTimeout(() => (this.status = ''), 3000);
}
```